

Claude Code Harness は、 結局なにをしてくれる道具なのか？

AI コーディングツールに「作業の型」と「安全の枠」を後付けする拡張機能です。v5.4.0 時点の実装を、使う人の視点から説明します。

人間が計画から検証まで見張る代わりに、AI が計画・実装・検証の材料を揃え、人間は最後の判断だけをする。その状態を、規則で機械的に強制する道具です。

版 v5.4.0 (Phase 126 適用済み)

作成 2026-07-29

対象 operator

位置づけ 派生物であり仕様の正本ではない

やさしく言うと

AI にコードを書かせると、たいてい 2 つのことで困ります。ひとつは「勝手に進みすぎて、気づいたら意図と違うものができている」。もうひとつは「危ないことをしないか心配で、結局ずっと張り付いて見ていることになる」。

Claude Code Harness（以下 CCH）は、この 2 つを別々の仕組みで解きます。

前者には**作業の型**を与えます。計画を先に文書へ固定し、実装し、独立した目でレビューし、リリースする。この 4 工程を飛ばせないようにします。

後者には**安全の柵**を与えます。危険な操作をコマンド実行の直前で機械的に止めます。

役割でたとえるなら、工事現場の監督と安全柵です。監督（作業の型）は

「図面なしで着工しない」「検査前に引き渡さない」と手順を守らせます。安全柵

（安全の柵）は、誰が何を言おうと立ち入り禁止区域に入れません。実際には、

監督にあたるのが 21 個のスキル（`/harness-plan` などの手順書）、安全柵にあたるのが 15 個の規則と 5 分類の最終防壁です。

押さえる本質は「AI を賢くする道具ではない」という点です。AI

の判断そのものには手を触れず、**その周りの手順と境界**だけを固めます。だから AI

のモデルが新しくなっても仕組みはそのまま使えます。

これを入れると何が変わる？

AI が「計画 → 実装 → レビュー → リリース」の順を飛ばせなくなります。加えて、外部への送信・秘密ファイルの読み取り・本番反映・作業場の外の削除・課金操作が、実行の直前で止まります。

毎回確認を求められて面倒では？

確認は計画を決める時点に前倒しします。「この作業では公開に push します」と一度承認しておけば、作業中は同じことで聞き直しません。承認には期限と回数の上限が付きます。

Claude 専用？

いいえ。Claude Code に加えて Codex CLI と Cursor が正式対応です。同じ 1 つの判定エンジンを、各ツールの標準的な差し込み口から呼ぶ構造なので、規則が枝分かれません。

導入コストは？

セットアップ 1 コマンドと、最初の計画づくり 1 回です。既存のリポジトリに後付けでき、コードの書き方や言語には制約を課しません。

何でできているか（実名と正体）

CCH は 4 種類の部品でできています。名前より「どこで効くか」を押さえてください。

構成要素と担当範囲

スキル（21 個）

`/harness-plan` のように呼び出す手順書。AI が読んで従う作業マニュアル。計画・実装・レビュー・リリースなど工程ごとにある。

エージェント（4 個）

実装役・レビュー役・助言役など、役割を分けた AI の設定。レビュー役は実装役の会話を引き継がないので、身内チェックにならない。

フック（27 種）

ツールの動作に割り込む差し込み口。コマンド実行の直前・直後、セッション開始時などに、後述の判定エンジンと呼ぶ。

判定エンジン（Go 製）

許可・確認・拒否を機械的に決める本体。規則 15 個と最終防壁 5 分類を持つ。全ツール共通で、ここだけが判断する。

押さえる本質は、判定が 1 か所に集まっていることです。ツールが 3 つあっても規則は 3 つに増えません。各ツールの差し込み口から同じエンジン呼びます。**AI に任せてよい詳細**は、スキルの中身やフックの配線です。使う側が読む必要はありません。

数えられるもの	現在	意味
スキル	21	工程ごとの手順書。中核は計画・実装・レビュー・リリース・初期設定の 5 つ
ガードレール規則	15	R01～R15。許可・確認・拒否を決める

最終防壁の分類	5	設定で無効化できない絶対の境界
自動テスト	1,587 + 188	Go の関数単位テストと、手順を通しで動かす検査

つながるとどう回るか（4 工程）

使う側が意識するのは 4 つのコマンドだけです。それぞれが次の工程に必要な材料を残します。

通常の 1 サイクル

1 計画 — `/harness-plan`

やりたいことを渡すと、仕様と作業一覧の 2 つの文書に落とします。各作業には「何をもって完了とするか」を検証可能な形で書きます。ここで **事前確認** も出ます。
この作業で発生しそうな危険操作を先にまとめて承認します。



2 実装 — `/harness-work`

作業一覧を上から実行します。件数に応じて、単独・並列・チーム編成を自動で選びます。
実装を別のツール（Codex や Cursor）に外注することもできます。動作が変わる変更では、先に失敗するテストを書く証拠を求めます。



3 レビュー — `/harness-review`

実装した本人とは別の目で差分を見ます。会話の履歴を共有しない独立したレビュー役が入り、必要なら別ツールの意見も取ります。重大な指摘が残っている限り完了になりません。



4 リリース — `/harness-release`

版番号の更新、変更履歴の整理、PR、タグ、公開までを続けて行います。人間の確認はこの工程で 1 回だけです。そこまでの道中では聞き直しません。

この順序は推奨ではなく、機械的に守らせます。

たとえばレビューで重大な指摘が残っていれば完了扱いにできませんし、リリース前に未レビューの実装があれば先にレビューへ回されます。

安全網は 2 層ある（役割が違う）

ここが CCH の中心です。強さも、外せるかどうかもう 2 層を重ねています。

コマンドが実行されるまでに通る 2 層

AI がコマンドを実行しようとする

ファイル書き込み、シェルコマンド、ファイル読み取りなど



1 最終防壁（5 分類） — 設定で外せない

課金操作 / 外部への送信 / 秘密ファイルの読み取り / 本番反映・公開 / 作業場の外の破壊。
該当したら即座に拒否します。権限を緩めるモードで動かしていても効きます。



2 ガードレール規則（R01～R15） — 一部は設定可能

保護されたファイルへの書き込み、強制 push、履歴の巻き戻し、秘密ファイルの誤コミットなど。
拒否するもの、確認を出すもの、警告だけのものがあります。



実行 / 確認ダイアログ / 拒否

どちらかで止まった場合、その事実が記録ファイルに 1 行残ります

なぜ 2 層に分けてあるか

用途が違うためです。**最終防壁**は「自律的に動く AI が暴走したときの最後の一線」で、どの設定からも到達できない独立した経路に置いてあります。

無効化する手段を構造的に持ちません。ここを緩めたいという要求は、原則として通らない設計です。

一方の**ガードレール規則**は日常の運用を調整する層です。たとえば「main への直接 push」は既定で確認を出しますが、設定で禁止にも許可にも変えられます。

プロジェクトの事情に合わせる余地はこちらにあります。

層	止め方	無効化	例
最終防壁	即座に拒否	できない	作業場の外の削除、秘密鍵の読み取り、本番デプロイ
ガードレール規則	拒否 / 確認 / 警告	一部は設定可能	保護ファイルの書き換え、強制 push、main への直接 push

確認を減らす仕組み（事前確認）

安全網を強くすると確認が増えます。CCH はこれを**時点をずらす**ことで解きます。
 作業中に何度も聞くのではなく、計画を決める時点で「この作業では公開に push します」とまとめて一度承認します。

承認には**期限・対象作業の一致・使用回数**の上限の 3 つが付きます。
 一度承認したものが永久に効き続けることはありません。また、
 この承認は最終防壁には効きません。緩められるのはガードレール規則の側だけです。

どこから使えるか

対応の度合いは正直に区別しています。「動くはず」ではなく、
検証の証拠がある範囲だけを対応と呼びます。

ツール	対応	実際にできること
Claude Code	正式対応	すべての工程。主たる開発対象
Codex CLI	正式対応	セットアップ script あり。作業の通し検証とリリース前検査が通っている
Cursor	正式対応	セットアップ script あり。作業の通し検証とリリース前検査が通っている
Codex アプリ版	候補	簡易検証のみ。CLI 版の実績は流用しない
OpenCode	内部互換	案内を出力する形。実行時の同等性は主張しない

「候補」と「内部互換」を対応と書かないのは、
後から食い違いが出たときに一番損をするのが使う側だからです。
この区別は文書上の建前ではなく、対応を格上げするには検証を通す必要があります。

普通に AI へ頼む場合と、何がどう違うか

場面	CCH なし	CCH あり
作業の始め方	チャットで依頼して即着手	仕様と作業一覧を先に文書化。 完了条件を検証可能な形で固定
レビュー	実装した AI が自分で確認	会話を共有しない独立した目が見る。 重大な指摘が残ると完了にできない
危険な操作	権限設定に依存。 緩めれば素通り	実行直前に機械判定。 最終防壁は権限設定を無視して効く
確認の回数	作業中に都度	計画時に一括。承認済みの範囲は作業中に聞かない
止まった理由	記録が残らない	規則名と判定を記録。後から集計できる
ツールの乗り換え	ツールごとに設定をやり直す	判定エンジンは共通。 対応ホストなら同じ規則が効く

費用対効果で見ると、効くのは**作業が長くなるほど**です。10分で終わる修正には手順が重すぎます。逆に、複数ファイルにまたがる変更や、AIに任せて席を外したい作業では、後戻りの回数が減る分だけ元が取れます。

今回の版で変わったこと（Phase 126）

「確認が多すぎて作業が進まない」という指摘から始まった改修です。
確認を減らす作業の途中で、逆に防御が効いていない箇所が 2 つ見つかったため、そちらを先に塞ぎました。

塞いだ穴

見つかった 2 件（いずれも実測で確認）

① 自律実行中の削除が無防備だった

削除コマンドの書き方を変えるだけで、作業場の外を消す操作が最終防壁をすり抜けていた。
受け止めていた確認の側は、自律実行中に無効になる設計だったため、防御が両方とも外れていた。

② 秘密ファイルの読み取りを迂回できた

複数行の文字列をシェルに渡す書き方の中に読み取りを隠すと、判定側が「ただの文書」とみなして中身を見ていなかった。実際にはシェルがそれを実行する。

減らした確認

場面	変更前	変更後
一時ファイルの書き込み	確認	通過
作業ツリー内の削除	確認	通過
承認済みの保護ブランチ push	確認	通過（承認が有効な間のみ）

緩めたのは「安全だと確定できる場合だけ」です。
削除の対象が実行時にしか決まらない書き方は、すべて確認が残ります。具体的には、
変数展開で対象が決まる、コマンド置換で後から作られる、削除の前に別のコマンドがある、
パイプラインやバックグラウンド実行、親ディレクトリ参照を含む、といった場合です。近道

(シンボリックリンク) は実体をたどってから判定するため、作業場の内側から外を指すリンク経由でも通りません。

記録が残るようになった

確認や拒否が起きるたびに、規則名・分類・判定を 1 行記録します。これにより「どの規則が何回止めたか」を後から数えられます。秘密情報は残しません。コマンドやファイルパスの生の文字列は書かず、ハッシュと長さだけを持ちます。秘密ファイル読み取りと課金操作の分類では、そのハッシュと長さすら省きます。

どこから始めるか

まず体験する

初期設定を 1 回走らせ、小さめの依頼で計画から実装まで一周する。手順の重さと効果を自分の作業で測る。

向いている場面: 導入するか決めていない

安全網だけ先に入れる

工程の型は使わず、危険操作の判定だけを効かせる。既存の進め方を変えずに、事故だけ減らす。

向いている場面: 手順を変えたくない

長時間の自律作業に使う

計画で承認をまとめて取り、席を外して回す。止まった箇所は記録から後で確認する。

向いている場面: まとまった量の作業がある

1 初期設定を実行する

なぜ: 対応ホストごとに差し込み口の配線が要するため。1 コマンドで済む

2 小さな依頼で計画を作る

なぜ: 仕様と作業一覧がどう書かれるかを見ると、以降の期待値が合う

3 事前確認の内容を読んでから承認する

なぜ: ここで承認した範囲は作業中に聞き直されない。承認の粒度を体感できる

4 1 件だけ実装させてレビューまで通す

なぜ: レビューが独立して働くかを、自分の目で確かめられる

5 止まった箇所があれば記録を見る

なぜ: どの規則が効きすぎているかを推測でなく実測で判断できる

CCH は「AI を賢くする道具」ではなく「AI に任せた作業の手順と境界を機械的に固定する道具」です。効果が出るのは、作業が長く、後戻りの損失が大きい場面に限られます。短い修正には手順が重すぎるため、常用より用途を選んで使うほうが費用対効果は高くなります。

正直な但し書き

- **これは仕様の正本ではありません。**実装は変わり続けます。判断の根拠に使う場合は、末尾の出典を直接あたってください。
- **数値は本文書の作成時点の実測です。**スキル 21 個、規則 15 個などは版によって変わります。
- **安全網は万能ではありません。**最終防壁が見ているのはシェルコマンドの文面です。判定できない書き方は「安全側に倒す」設計ですが、それは「すべてを検出できる」という意味ではありません。
- **確認を緩める設定には副作用があります。**たとえば外部送信の制限を切ると、作業場の中で読んだ情報を外へ送る経路が理論上は開きます。人が見ている前提での運用かどうかで判断が変わります。
- **修正はプラグイン更新まで反映されません。**本体を直してもすぐ効くわけではなく、更新して再読み込みが必要です。

出典 (SSOT)

- `spec.md` — Purpose / North Star (3 層) / Risk Gate distribution contract
- `docs/spec/operations-memory-and-collaboration.md` — 最終防壁 5 分類と非上書き契約、auto-approve scope
- `README.md` — ホスト対応 tier (Codex CLI / Cursor = supported、Codex app = candidate、OpenCode = internal-compatible)
- `go/internal/policy/rules.go` — ガードレール規則 R01~R15
- `go/internal/runtimefloor/runtimefloor.go` — 最終防壁の実装
- `CHANGELOG.md` [Unreleased] — Phase 126 の変更内容
- `Plans.md` Phase 126 — 各作業の完了条件と検証記録

分析方法と制約

- 数値はリポジトリ実測（スキル 21、エージェント 4、規則 15、フック 27 種、Go テスト 1,587、シェル検査 188、規約 21）。
- Phase 126 の変更前後の挙動は、判定エンジンに直接入力をして観測した実測値。テスト結果の引用ではない。
- ホスト対応 tier は README の記載をそのまま採用。本文書では格上げも格下げもしていない。
- 費用対効果の記述（短い修正には重い等）は運用上の判断であり、計測に基づく数値ではない。

本レポートは派生物であり、仕様の正本ではありません。作成 2026-07-29 / claude-code-harness v5.4.0。